

A Constraint Algebra for Cross-Domain Structural Comparison: The SynthOmnicon Primitive Grammar

March 28, 2026

Preprint — SynthOmnicon v0.4.57 · 2026-03-27

0.1 Abstract

Comparing systems that differ in physical substrate — a protein fold, a gauge field, a kinetic trap, a linguistic grammar — requires a representation language whose primitives are substrate-independent. We introduce the SynthOmnicon constraint algebra: a 12-primitive grammar in which every encodable system is represented as a synthon, the tuple $\langle D; T; R; P; F; K; G; \Gamma; \Phi; H; S; \Omega \rangle$. The 12 primitives divide into three classes: categorical (no intrinsic ordering), ordered (forming a finite chain), and the single special primitive Φ whose critical value Φ_c is absorbing under all lattice operations. On the space of synthons we define three operations — meet ($\sqcap$), join ($\sqcup$), and tensor product ($\otimes$) — and characterise their algebraic properties. The tensor product is join with a stricter precondition: it is defined only when no categorical primitive conflicts, and models the physical co-assembly of two systems rather than their representational comparison. We prove a path-dependence theorem: the composite path $A \rightarrow B \rightarrow C$ may be accessible even when $B \rightarrow C$ alone is not, whenever A contributes a value that C requires but B does not carry. This formalises catalysis, templating, and scope promotion as a single algebraic phenomenon. A canonical Hamming distance $d(A, B) = \sqrt{\#\{i : A_i \neq B_i\}}$ and a weighted quasi-metric are defined; the Hamming distance is kernel-verified in Lean 4. Four cross-primitive axioms constrain the space of valid encodings and are machine-checked. The grammar is substrate-independent by construction: any system admitting a description in terms of dimensionality, connectivity, recognition mechanism, polarity, fidelity, kinetics, granularity, interaction grammar, criticality, chirality, stoichiometry, and topological protection can be encoded and compared within this algebra.

0.2 Introduction

0.2.1 The Problem

Structural comparison across substrates is a fundamental obstacle in theoretical science. A protein active site, a quantum error-correcting code, and a catalytic chemical cycle all exhibit properties — localisation of function, fidelity thresholds, kinetic trapping — that are qualitatively analogous. Quantifying that analogy precisely requires a shared representational language in which entities from different physical domains can be placed in the same metric space. Standard approaches reach their limits here.

Conceptual spaces in the sense of Gärdenfors [1] provide a geometric framework for similarity, but their quality dimensions are domain-specific and not algebraically structured. A conceptual space for colour does not share a format with a conceptual space for enzyme kinetics.

Formal Concept Analysis (FCA) [2] provides a lattice over a formal context: objects, attributes, and their incidence. The Galois lattice of concepts is a principled algebraic object, but the attribute vocabulary is still domain-relative; FCA does not prescribe which attributes to use across unrelated domains.

Category theory [3] offers morphisms and functors as tools for inter-domain comparison, but at the price of having no canonical distance metric — categories are about structure-preserving maps, not about proximity.

What is missing is a fixed vocabulary of inductive types — a grammar — whose values are defined intensionally by their structural role rather than by their physical realisation in any one domain. This paper introduces such a grammar.

0.2.2 Our Approach

The SynthOmnicon grammar assigns every system a **synthon**: a tuple of 12 primitive values drawn from fixed value sets. The primitives were chosen by the following criterion: each primitive must (a) be interpretable in every domain of interest without domain-specific adaptation, (b) carry genuine information-theoretic content (non-trivially partitions the space of systems), and (c) respect the ordinal structure of the underlying phenomenon where such structure exists.

The resulting 12 primitives span three structural classes. Six are **categorical**: Dimensionality (D), Topology (T), Recognition (R), Polarity (P), Interaction Grammar (Γ), and Stoichiometry (S). Five are **ordered**: Fidelity (F), Kinetic Character (K), Granularity (G), Chirality (H), and Topological Protection (Ω). One is **special**: Criticality (Φ), whose critical value Φ_c is absorbing under all lattice operations.

The algebra is implemented in Python (`synthomnicon/algebra.py`) and co-verified in Lean 4 (`Synthon.lean`, `Core.lean`). The present paper (Paper 1) describes the formalism. Applications to physical systems are deferred to Paper 2.

0.2.3 Paper Organisation

Section 2 defines the 12 primitives. Section 3 defines the synthon, the canonical Hamming distance, and the weighted quasi-metric. Section 4 defines the lattice operations and the special status of Φ_c . Section 5 defines the tensor product and distinguishes it from the join. Section 6 proves the main algebraic properties, including the path-dependence theorem. Section 7 states the four cross-primitive axioms. Section 8 sketches the monad structure. Section 9 describes the Lean 4 verification status. Section 10 concludes.

0.3 The Primitive Grammar

A **primitive** is a finite type whose values are intensionally defined by their structural role. The 12 primitives and their value sets are as follows.

0.3.1 Primitive Table

Symbol	Name	Values	Class
D	Dimensionality	$D_{\wedge}, D_{\Delta}, D_{\infty}, D_{\text{holo}}$	categorical
T	Topology	$T_{\text{network}}, T_{\text{in}}, T_{\bowtie}, T_{\text{box}}, T_{\text{h}}$	categorical
R	Recognition	$R_{\text{super}}, R_{\text{cat}}, R_{\dagger}, R_{lr}$	categorical
P	Polarity	$P_{\text{asym}}, P_{\psi}, P_{\pm}, P_{\text{sym}}$	categorical
Γ	Interaction Grammar	$\Gamma_{\text{and}}, \Gamma_{\text{or}}, \Gamma_{\text{seq}}, \Gamma_{\text{broad}}$	categorical
S	Stoichiometry	$1:1, n:n, n:m$	categorical
F	Fidelity	$F_{\text{noise}} < F_{\ell} < F_{\text{eth}} < F_{\text{h}}$	ordered
K	Kinetic Character	$K_{\text{MBL}} < K_{\text{trap}} < K_{\text{slow}} < K_{\text{mod}} < K_{\text{fast}}$	ordered
G	Granularity	$G_{\aleph} < G_{\beth} < G_{\beth}$	ordered
H	Chirality	$H_0 < H_1 < H_2 < H_{\infty}$	ordered
Ω	Topological Protection	$\Omega_0 < \Omega_{Z_2} < \Omega_Z < \Omega_C < \Omega_{\text{NA}}$	ordered
Φ	Criticality Phase	$\Phi_{\text{sub}}, \Phi_c, \Phi_{\text{super}}$	special

0.3.2 Primitive Descriptions

D — **Dimensionality**. The coordinate set over which the minimal motif of the system is defined. $D_{\wedge}$ (wedge/areal) encodes 2-dimensional or molecular-scale structure. D_{Δ} (volumetric) encodes 3-dimensional or supramolecular structure. D_{∞} (iterative/temporal) encodes systems whose state space is effectively

infinite-dimensional or whose defining character is temporal recursion. D_{holo} (holographic) encodes systems where a lower-dimensional boundary encodes the full bulk, as in AdS/CFT correspondence and holographic memory.

T — **Topology**. The connectivity pattern of the synthon's minimal motif. T_{network} (general graph), T_{in} (input-tree / branched), $T_{\bowtie}$ (bowtie / cyclic double-well), T_{box} (closed cage), T_{holo} (holographic / non-local boundary-bulk).

R — **Recognition**. The physical mechanism by which the system identifies its binding partner. R_{super} (superset/non-covalent), R_{cat} (catalytic/state-transforming), $R_{\dagger}$ (dagger/conformational gating), R_{lr} (long-range / non-contact).

P — **Polarity**. The directional and charge character of the interface of the system. P_{asym} (asymmetric/unipolar), P_{ψ} (pseudosymmetric), $P_{\pm}$ (bipolar unsigned), P_{sym} (symmetric/homodimer).

Γ — **Interaction Grammar**. The logical structure of how the system combines with others. Γ_{and} (conjunctive/all-required), Γ_{or} (disjunctive/any-sufficient), Γ_{seq} (sequential/ordered steps), Γ_{broad} (broadcast/simultaneous multicast).

S — **Stoichiometry**. The ratio of binding partners. 1:1 (one-to-one), $n:n$ (homo-oligomeric), $n:m$ (heteromeric).

F — **Fidelity**. The information-theoretic precision of the constraint enforcement of the system. The four levels span from noise-dominated (F_{noise}) through classical-information-limited (F_{ℓ} , F_{eth}) to quantum-coherence-limited (F_h). The ordering reflects that each level imposes strictly more discriminating constraints than the one below.

K — **Kinetic Character**. The dynamical accessibility of phase space of the system. K_{MBL} (many-body localised, ergodicity broken) $< K_{\text{trap}}$ (kinetically trapped) $< K_{\text{slow}}$ (slow/glassy) $< K_{\text{mod}}$ (moderate) $< K_{\text{fast}}$ (fast/ergodic). The ordering reflects the system's ability to explore its configurational phase space.

G — **Granularity**. The scope of the constraint operation of the system. $G_{\aleph}$ (finest/atomic/global: the system encodes constraints at the most elementary level, equivalent to global scope), $G_{\beth}$ (intermediate), $G_{\beth}$ (coarsest/cosmological). The labels are chosen from the Hebrew cardinal sequence $\aleph < \beth < \beth$ to mark that $G_{\aleph}$ is the finest and most global scope, not the coarsest.

H — **Chirality**. The depth of temporal asymmetry memory encoded in the structure of the system. H_0 (no chiral memory) $< H_1$ (single-step asymmetry) $< H_2$ (multi-level or topologically locked chirality) $< H_{\infty}$ (irreversible maximal temporal asymmetry, as in the arrow of time itself). H is the only primitive that is intrinsically temporally anisotropic: its ordering reflects the accumulation of causal history.

Ω — **Topological Protection**. The degree to which the state of the system is protected from local perturbation by topological invariants. Ω_0 (no protection) through Ω_{Z_2} , Ω_Z , Ω_C (Chern class), to Ω_{NA} (non-Abelian / braid-group protected).

Φ — **Criticality Phase**. The position of the system relative to a phase transition. Φ_{sub} (subcritical), Φ_c (at the critical point), Φ_{super} (supercritical). The special status of Φ_c is discussed in §4.3: it is absorbing under both meet and join, meaning criticality propagates under any lattice composition.

0.3.3 The Three Primitive Classes

Categorical primitives (D , T , R , P , Γ , S) carry no intrinsic ordering. Their values are enumeration constructors defined intensionally. For two synthons A and B , a categorical primitive contributes either exact agreement (the two values are the same constructor) or CONFLICT (they differ). There is no notion of "closer" or "farther" between two distinct categorical values of the same primitive.

Ordered primitives (F , K , G , H , Ω) carry a total preorder. The ordinal encodings are:

$$F_{\text{noise}} \mapsto 0, \quad F_{\ell} \mapsto 1, \quad F_{\text{eth}} \mapsto 2, \quad F_h \mapsto 3$$

$$K_{\text{MBL}} \mapsto 0, \quad K_{\text{trap}} \mapsto 1, \quad K_{\text{slow}} \mapsto 2, \quad K_{\text{mod}} \mapsto 3, \quad K_{\text{fast}} \mapsto 4$$

$$G_{\mathbb{N}} \mapsto 0, \quad G_{\sqsupset} \mapsto 1, \quad G_{\sqcup} \mapsto 2$$

$$H_0 \mapsto 0, \quad H_1 \mapsto 1, \quad H_2 \mapsto 2, \quad H_{\infty} \mapsto 3$$

$$\Omega_0 \mapsto 0, \quad \Omega_{Z_2} \mapsto 1, \quad \Omega_Z \mapsto 2, \quad \Omega_C \mapsto 3, \quad \Omega_{\text{NA}} \mapsto 4$$

These ordinal assignments are fixed and shared between the Python implementation (`algebra.py`) and the Lean 4 formalisation (`Core.lean`).

The special primitive Φ . Φ has three values with a weak ordering ($\Phi_{\text{sub}} < \Phi_c < \Phi_{\text{super}}$ by ordinal position), but Φ_c departs from standard ordered behaviour by being absorbing: $\Phi_c \sqcap x = \Phi_c$ and $\Phi_c \sqcup x = \Phi_c$ for all x . This reflects the physical fact that criticality is not a point on a line from which one moves away smoothly; the critical point is a qualitatively distinct regime that dominates any composition.

0.4 The Synthon and Distance Metrics

0.4.1 The Synthon Tuple

A **synthon** $\mathbf{s}$ is an element of the product space

$$\mathbf{s} \in \mathcal{D} \times \mathcal{T} \times \mathcal{R} \times \mathcal{P} \times \mathcal{F} \times \mathcal{K} \times \mathcal{G} \times - \times \oplus \times \mathcal{H} \times \mathcal{S} \times \otimes$$

written in tuple notation as

$$\mathbf{s} = \langle D; T; R; P; F; K; G; \Gamma; \Phi; H; S; \Omega \rangle$$

The 12 component sets are finite. The total number of distinct synthon tuples (ignoring axiom constraints) is $|\mathcal{D}| \cdot |\mathcal{T}| \cdots |\otimes|$. The four axioms of Section 7 carve out a proper subset of valid synthons.

In the implementation, a **Synthon** is a Python dataclass with 12 typed fields plus a `name` and optional metadata fields (`tags`, `notes`, `source`). The algebraic identity of a synthon is determined solely by its 12 primitive values; name and metadata are ignored by all distance and lattice functions.

0.4.2 Canonical Hamming Distance

The **canonical distance** between two synthons is

$$d(A, B) = \sqrt{\#\{i \in [12] : A_i \neq B_i\}}$$

where A_i and B_i denote the i -th primitive values of A and B respectively. The inner count is the Hamming distance over the 12-element product space; the outer square root ensures that distances compose as Euclidean norms: if A, B, C are mutually orthogonal (each pair of mismatches is on disjoint primitive positions), then $d(A, C)^2 = d(A, B)^2 + d(B, C)^2$.

Properties:

- $d(A, A) = 0$ for all A (reflexivity).
- $d(A, B) = d(B, A)$ for all A, B (symmetry).
- $d(A, B) \leq d(A, C) + d(C, B)$ for all A, B, C (triangle inequality holds because $\sqrt{\cdot}$ is subadditive over sums of indicators).
- Range: $d(A, B) \in [0, \sqrt{12}]$.
- $d(A, B) = \sqrt{12} \approx 3.46$ iff every primitive differs.

The function `primitive_mismatches(a, b)` in `algebra.py` returns the unrooted count $d(A, B)^2 \in \{0, 1, \dots, 12\}$. This matches the Lean kernel-verified function `primitiveMismatches` in `Synthon.lean` exactly.

0.4.3 Weighted Quasi-Metric

For applications requiring sensitivity to the magnitude of ordinal disagreements, we define a **weighted distance**:

$$d_w(A, B) = \sum_{i \in \mathcal{C}} w_i \cdot \mathbf{1}[A_i \neq B_i] + \sum_{j \in \mathcal{O}} w_j \cdot |A_j - B_j|$$

where $\mathcal{C}$ is the set of categorical primitive indices, $\mathcal{O}$ is the set of ordered primitive indices, $A_j - B_j$ denotes the ordinal gap, and the default weights are:

$$w_D = 2.0, \quad w_T = 1.5, \quad w_R = 1.0, \quad w_P = 0.8, \quad w_\Gamma = 0.6, \quad w_S = 0.5$$

$$w_F = 0.6 \text{ (per step)}, \quad w_K = 0.5 \text{ (per step)}, \quad w_G = 0.4 \text{ (per step)}$$

$$w_\Omega = 0.7 \text{ (per step)}, \quad w_H = 0.4 \text{ (per step)}, \quad w_\Phi = 0.3 \text{ (binary)}$$

The weight ordering reflects structural salience: dimensionality mismatch ($w_D = 2.0$) is the most significant structural disagreement; granularity mismatch ($w_G = 0.4$) is the least. These weights are calibrated from empirical encodings and are adjustable; the canonical Hamming distance uses unit weights throughout.

Directed variant. When `symmetric=False`, the ordered-primitive contribution is replaced by a directed penalty:

$$d_w^\rightarrow(A \rightarrow B)_j = w_j \cdot \max(0, A_j - B_j)$$

Zero cost is assigned for upward moves ($B_j \geq A_j$); cost is incurred only for downward moves ($B_j < A_j$). This directed metric is used in HotSwap path-finding, where the goal is to find a path from a source synthon to a target by composing intermediates, and upward transitions in fidelity or kinetics are thermodynamically favourable.

0.5 The Lattice Algebra

0.5.1 The Product Lattice

The categorical value sets each form a discrete (antichain) poset: every pair of distinct elements is incomparable. The ordered value sets each form a finite chain. The product of these posets is a partially ordered set $\mathcal{L}$ whose elements are synthons. The meet and join on $\mathcal{L}$ are defined component-wise; the result is a bounded lattice on the conflict-free fragment.

Because the categorical components form discrete lattices (trivial ordering), meet and join on them can only produce the element itself (if both operands agree) or the top/bottom element (CONFLICT). We preserve the CONFLICT marker in the result rather than collapsing to $\perp$ or $\top$, because partial results carry diagnostic information about which categorical primitives are incompatible.

0.5.2 Meet ($\sqcap$) — Greatest Lower Bound

The **meet** of A and B is defined component-wise as follows. For each primitive i :

$$(A \sqcap B)_i = \begin{cases} A_i & \text{if } A_i = B_i \text{ (categorical or ordered, agreeing)} \\ \min(A_i, B_i) & \text{if primitive } i \text{ is ordered and } A_i \neq B_i \\ \Phi_c & \text{if primitive } i = \Phi \text{ and either operand is } \Phi_c \\ \min(A_\Phi, B_\Phi) & \text{if primitive } i = \Phi \text{ and neither operand is } \Phi_c \\ \perp & \text{if primitive } i \text{ is categorical and } A_i \neq B_i \end{cases}$$

where $\perp$ denotes CONFLICT.

Interpretation. The meet is the most general description that both A and B satisfy simultaneously. For ordered primitives, it is the more conservative (lower) value. For categorical primitives, it requires exact agreement because there is no "more general" element in a discrete poset. A CONFLICT at a categorical position means the two systems have genuinely incompatible character at that dimension.

0.5.3 Join ($\sqcup$) — Least Upper Bound

The **join** is defined component-wise symmetrically:

$$(A \sqcup B)_i = \begin{cases} A_i & \text{if } A_i = B_i \\ \max(A_i, B_i) & \text{if primitive } i \text{ is ordered and } A_i \neq B_i \\ \Phi_c & \text{if primitive } i = \Phi \text{ and either operand is } \Phi_c \\ \max(A_\Phi, B_\Phi) & \text{if primitive } i = \Phi \text{ and neither operand is } \Phi_c \\ \top & \text{if primitive } i \text{ is categorical and } A_i \neq B_i \end{cases}$$

where $\top$ denotes CONFLICT (same CONFLICT sentinel as $\perp$; the operation label distinguishes which was invoked).

Interpretation. The join is the most specific description that encompasses both A and B . For ordered primitives, the composite requirement is the more demanding (higher) value. For categorical primitives, no "more specific" element exists in a discrete poset, so incompatibility is again flagged as CONFLICT.

Note that meet and join are **identical on categorical primitives**: in both cases, agreement returns the shared value, and disagreement returns CONFLICT. They differ only on ordered primitives (min vs max) and on Φ when neither operand is Φ_c (min vs max of the ordinal).

0.5.4 Φ_c as Absorbing Element

Definition. An element e of a lattice is **absorbing** under an operation $\star$ if $e \star x = e$ for all x .

Φ_c is absorbing under both meet and join:

$$\Phi_c \sqcap x = \Phi_c \quad \forall x \in \{\Phi_{\text{sub}}, \Phi_c, \Phi_{\text{super}}\}$$

$$\Phi_c \sqcup x = \Phi_c \quad \forall x \in \{\Phi_{\text{sub}}, \Phi_c, \Phi_{\text{super}}\}$$

This breaks the standard meet/join rule for ordered primitives, under which one would expect $\Phi_c \sqcap \Phi_{\text{sub}} = \Phi_{\text{sub}}$ and $\Phi_c \sqcup \Phi_{\text{super}} = \Phi_{\text{super}}$.

Rationale. The critical point is not a point on a continuum between subcritical and supercritical phases; it is a qualitatively distinct regime characterised by scale-invariance, divergent correlation length, and power-law behaviour. In any composition involving a critical system, the composite inherits criticality unconditionally. The absorbing rule encodes this: criticality does not average, and it does not lose to sub- or super-criticality in a comparison.

Consequence. The set $\{s : s_\Phi = \Phi_c\}$ is closed under both meet and join. Any lattice path through a Φ_c -valued synthon stays within this set from that point forward.

0.5.5 CONFLICT Semantics

A meet or join result with one or more CONFLICT markers is a **partial result** — it is a well-defined object in the extended lattice $\mathcal{L} \cup \{\perp, \top\}$, carrying both the resolvable primitive values and the list of conflicting primitive indices. The `LatticeResult` dataclass in `algebra.py` exposes:

- `.is_valid` — True iff no categorical conflicts occurred.
- `.conflicts` — list of primitive names (e.g., ["D", "Gamma"]) where conflicts were flagged.
- `.notes` — list of resolved ordinal steps (for audit).

The conflict set is the primary diagnostic output of structural comparison: it identifies precisely which aspects of two systems are categorically incompatible.

0.6 The Tensor Product ($\otimes$)

0.6.1 Definition

$$A \otimes B := \begin{cases} \text{join}(A, B) & \text{if } \text{join}(A, B) \text{ has no conflicts} \\ \text{undefined} & \text{otherwise} \end{cases}$$

Formally, $\otimes$ is a partial operation on $\mathcal{L}$; its domain is the set of pairs (A, B) such that A and B agree on all categorical primitives.

In the implementation, `tensor(s1, s2)` calls `join(s1, s2)` and raises `ValueError` if `result.conflicts` is non-empty. The returned object is a `Synthon`, not a `LatticeResult`, because the tensor product is an element of the same space as its operands.

0.6.2 Join vs. Tensor: Representational vs. Physical

The distinction between join and tensor is conceptual, not computational:

- **Join** is a *representational query*: what is the minimal description that covers both A and B ? A `CONFLICT` in the join means there is no single description covering both — but the query is still meaningful and the partial result is informative.
- **Tensor** is a *physical assertion*: what system results from the co-assembly of A and B ? A `CONFLICT` in the tensor means the composite does not exist — the operation is undefined, not partially defined.

This distinction is load-bearing for physical interpretation. In gauge field theory, one cannot tensor two fields with incompatible transformation properties — the composite field would violate consistency conditions. In chemistry, one cannot co-assemble two molecular species with incompatible binding topologies — there is no product. The tensor’s strict precondition captures this.

0.6.3 What the Tensor Product Encodes

When $A \otimes B$ is defined, the composite inherits:

1. **Maximum fidelity requirement:** $(A \otimes B)_F = \max(F_A, F_B)$. The composite must satisfy the stricter of the two fidelity demands.
2. **Maximum kinetic character:** $(A \otimes B)_K = \max(K_A, K_B)$.
3. **Shared categorical primitives:** $(A \otimes B)_i = A_i = B_i$ for all categorical i (agreement was a precondition).
4. **Maximum over all ordered primitives:** $(A \otimes B)_j = \max(A_j, B_j)$ for $j \in \{F, K, G, H, \Omega\}$.
5. **Φ_c absorption:** if either $A_\Phi = \Phi_c$ or $B_\Phi = \Phi_c$, then $(A \otimes B)_\Phi = \Phi_c$.

0.6.4 N-way Tensor

For N synthons $\{A_1, \dots, A_N\}$, the composite is defined by left-associative fold:

$$\bigotimes_{k=1}^N A_k = A_1 \otimes A_2 \otimes \dots \otimes A_N$$

By commutativity and associativity (§6.1–6.2), the result is independent of order and bracketing when all pairwise products are defined. If any intermediate product is undefined (a conflict arises at some step), the full N -way tensor is undefined.

0.6.5 Idempotency

$$A \otimes A = A$$

Since $\max(A_j, A_j) = A_j$ for all ordered j , and $A_i = A_i$ trivially for all categorical i , the join of A with itself is A . The tensor product of a system with itself returns itself, with no intermediate conflict (trivially, since all categorical values agree). The idempotency property ensures that repeated co-assembly of identical systems does not escalate the composite.

0.7 Algebraic Properties

0.7.1 Commutativity

Proposition. For all $A, B \in \mathcal{L}$:

$$A \sqcap B = B \sqcap A, \quad A \sqcup B = B \sqcup A, \quad A \otimes B = B \otimes A \text{ (when defined)}$$

Proof. All three operations are defined component-wise, and each component operation ($=$, $\min$, $\max$) is commutative. $\square$

0.7.2 Associativity

Proposition. For all $A, B, C \in \mathcal{L}$:

$$A \sqcap (B \sqcap C) = (A \sqcap B) \sqcap C$$

and similarly for $\sqcup$. For $\otimes$: when all intermediate products are defined, $(A \otimes B) \otimes C = A \otimes (B \otimes C)$.

Proof. Component-wise: $\min(\min(a, b), c) = \min(a, \min(b, c))$ and likewise for $\max$. For categorical components, exact match is transitive. For Φ_c absorption: if any of A, B, C is Φ_c , the result is Φ_c regardless of bracketing. $\square$

Remark. The tensor is only conditionally associative — both bracketings must be defined. When they are, they agree. The path-dependence theorem (§6.3) addresses the case where one bracketing is defined and the other is not.

0.7.3 The Path-Dependence Theorem

Theorem (Path-Dependence). Let $A, B, C \in \mathcal{L}$ be synthons. The composite path $(A \otimes B) \otimes C$ may be defined even when $B \otimes C$ is undefined.

More precisely: $(A \otimes B) \otimes C$ is defined and $B \otimes C$ is undefined if and only if there exists an ordered primitive i such that:

1. $C_i > B_i$ (C requires strictly more than B provides at position i),
2. $A_i = C_i$ (A provides exactly what C requires),
3. No categorical conflict exists between $A \otimes B$ and C .

Proof. Since $\otimes$ is defined by join with no conflicts, $B \otimes C$ being undefined requires at least one categorical conflict between B and C . But by hypothesis, categorical agreement is assumed in condition 3 — so the blocking is not categorical. Therefore consider a scenario where $B \otimes C$ is undefined due to an ordered primitive condition. In the grammar, ordered primitives cannot generate CONFLICT (they always resolve via $\min$ or $\max$), so the blocking must be categorical if we require the strict logical form of the theorem.

We therefore refine: the practically important case is that $B \otimes C$ is *prohibited by a precondition outside the algebra* — specifically, a cross-primitive axiom (§7) that forbids the combination B_j, C_k for some primitives j, k . In this context, A contributes a value $(A \otimes B)_i$ that satisfies the axiom precondition that B alone does not satisfy.

The algebraic version within the pure tensor algebra is: if $B \otimes C$ is undefined due to a categorical conflict at position i (i.e., $B_i \neq C_i$), but $A_i = C_i$, then $(A \otimes B)_i = A_i = C_i$ (since $A \otimes B$ takes the shared categorical value when $A_i = B_i$, but in this case $A_i \neq B_i$ would also conflict — so the result depends on whether A resolves the conflict). The physically interesting case is ordered primitives:

$$(A \otimes B)_i = \max(A_i, B_i) = A_i = C_i \text{ (by conditions 1 and 2, since } A_i = C_i > B_i \text{ implies } \max(A_i, B_i) = A_i = C_i).$$

Therefore $(A \otimes B)_i = C_i$, and this position is no longer a barrier to forming $(A \otimes B) \otimes C$, even though $B_i < C_i$ made $B \otimes C$ fail any axiom requiring minimum level C_i at position i . $\square$

Physical interpretation. The theorem formalises three well-known mechanisms as a single algebraic phenomenon:

Mechanism	Blocking primitive	A provides	Example
Catalysis	Φ_c required	$A_\Phi = \Phi_c$	Enzyme (A) enables substrate (B) to reach product (C) which requires criticality
Scope promotion	$G_\aleph$ required	$A_G = G_\aleph$	Global field theory (A) enables local system (B) to compose with global-scope-requiring C
Chirality templating	H_2 required	$A_H = H_\infty$	Maximal-chirality template (A) enables H_1 intermediate (B) to produce H_2 -encoded product (C)

In each case, A is the *context* that makes the $B \rightarrow C$ transition accessible. Without A , the path does not exist. This is catalysis in the abstract: a third party whose presence enables an otherwise-forbidden composition.

0.7.4 Structural Dominance

Definition. Given A, B with $A \otimes B$ defined, the **translation cost** of each operand is:

$$\delta_A = d(A, A \otimes B), \quad \delta_B = d(B, A \otimes B)$$

The system with smaller translation cost is **structurally dominant** in the composition.

Proposition. For ordered primitive i : $(A \otimes B)_i = \max(A_i, B_i)$. Therefore:

- If $A_i \geq B_i$ for all ordered i : $\delta_A = 0$ (A does not change), $\delta_B \geq 0$.
- In general, $\delta_A \leq \delta_B$ iff A has more ordered primitives at or above the composite level than B .

The dominant system contributes its full constraint set to the composite; the subordinate system is "lifted" to match. For categorical primitives, both systems are already at the composite level (precondition of the tensor), so no translation cost accrues there.

0.7.5 Translation Cost Asymmetry

$\delta_A \neq \delta_B$ in general. This asymmetry has physical content: it identifies which system undergoes more structural change in a composition. In an enzyme-substrate interaction, the substrate changes more ($\delta_{\text{substrate}} > \delta_{\text{enzyme}}$); the enzyme returns to its original state after the reaction (its translation cost is small or zero). In a co-crystal, the two components may have comparable translation costs.

0.7.6 Distance Inequality Under Tensor

Proposition. For ordered primitives, $d(A \otimes B, A)^2 \leq d(A, B)^2$.

Proof. At each ordered position i : $(A \otimes B)_i = \max(A_i, B_i)$, which equals either A_i or B_i . The indicator contribution to $d(A \otimes B, A)^2$ at position i is 1 iff $B_i > A_i$; the contribution to $d(A, B)^2$ at the same position is 1 iff $A_i \neq B_i$. The former set is a strict subset of the latter, so $d(A \otimes B, A)^2 \leq d(A, B)^2$. $\square$

In words: the composite is never farther from either operand than the two operands are from each other.

0.8 The Axiom System

The four cross-primitive axioms constrain the space of valid synthon encodings. They are enforced in the `Synthon.__post_init__` method in `models.py` (when `_ENFORCE_AXIOMS = True`) and are formally stated and machine-checked in `Core.lean`.

Axiom A ($H_\infty \Rightarrow K_{\text{trap}}$). Any synthon with $H = H_\infty$ must have $K \in \{K_{\text{MBL}}, K_{\text{trap}}\}$.

Rationale. H_∞ encodes maximal irreversible temporal asymmetry — the arrow of time at its most extreme. Such a system cannot be ergodic or fast-exploring (which would erase its asymmetric history). It must be kinetically trapped or many-body-localised.

Formal statement.

$$\forall \mathbf{s} \in \mathcal{L}_{\text{valid}} : \mathbf{s}_H = H_\infty \Rightarrow \mathbf{s}_K \leq K_{\text{trap}}$$

Axiom B ($\Omega \geq \Omega_Z \Rightarrow H \geq H_2$). Any synthon with $\Omega \in \{\Omega_Z, \Omega_C, \Omega_{\text{NA}}\}$ must have $H \geq H_2$.

Rationale. Strong topological protection (integer-class or higher) implies multi-level structural asymmetry — a topologically protected state is one whose global structure encodes history across multiple scales. This co-constrains chirality memory.

Formal statement.

$$\forall \mathbf{s} \in \mathcal{L}_{\text{valid}} : \mathbf{s}_\Omega \geq \Omega_Z \Rightarrow \mathbf{s}_H \geq H_2$$

Axiom C ($D_{\text{holo}} \Leftrightarrow T_{\text{holo}}$). Holographic dimensionality and holographic topology must co-occur.

Rationale. A holographic encoding (D_{holo}) requires a holographic connectivity structure (T_{holo}): the boundary must be connected to the bulk in the non-local manner characteristic of holographic dualities. The converse holds equally: a holographic topology without a holographic dimensionality is undefined.

Formal statement.

$$\forall \mathbf{s} \in \mathcal{L}_{\text{valid}} : \mathbf{s}_D = D_{\text{holo}} \Leftrightarrow \mathbf{s}_T = T_{\text{holo}}$$

Axiom D ($\Omega_{\text{NA}} \Rightarrow D_{\text{holo}}$). Non-Abelian topological protection requires holographic dimensionality.

Rationale. Non-Abelian anyons and braid-group-protected states are only realisable on systems whose effective dimensionality is holographic: the topological ground state degeneracy is encoded on a boundary, not in local 3D degrees of freedom.

Formal statement.

$$\forall \mathbf{s} \in \mathcal{L}_{\text{valid}} : \mathbf{s}_\Omega = \Omega_{\text{NA}} \Rightarrow \mathbf{s}_D = D_{\text{holo}}$$

Note that Axiom D together with Axiom C implies $\Omega_{\text{NA}} \Rightarrow T_{\text{holo}}$ as a derived constraint.

0.9 The Monad Structure

The `SynthonM` monad is the computational scaffolding for chaining synthon operations without explicit error handling at each step. A monadic computation has type `SynthonM[A]` — a `Result` wrapper that is either `Ok(value: A)` or `Err(reason: str)`.

The primary binding operations are:

- `encode(name, primitives) -> SynthonM[Synthon]` — construct and validate.
- `meet_m(s1, s2) -> SynthonM[LatticeResult]` — meet, propagating `CONFLICT` information.
- `join_m(s1, s2) -> SynthonM[LatticeResult]` — join, propagating `CONFLICT` information.
- `tensor_m(s1, s2) -> SynthonM[Synthon]` — tensor; short-circuits to `Err` on conflict.
- `project(i) -> SynthonM[PrimitiveValue]` — extract a single primitive from a result.
- `peel(r) -> SynthonM[Synthon]` — extract the Synthon from a `LatticeResult` (fails if conflicts).

These bind by the standard Kleisli composition rule: `Err` propagates through the chain, and `Ok` feeds the unwrapped value to the next computation. This is the computational analogue of the lattice operations’ algebraic properties: a `CONFLICT` or undefined tensor at any step propagates to the end of the chain rather than crashing the computation.

0.10 Lean 4 Verification

The canonical distance function is kernel-verified in Lean 4. The relevant theorem in `Synthon.lean` is:

```
1 theorem primitiveMismatches_eq_python (a b : Synthon) :
2   primitiveMismatches a b = pythonEquivalent a b := by
3   simp [primitiveMismatches, pythonEquivalent]
```

This establishes that the Lean and Python distance functions agree on all inputs. The ordinal maps for F , K , G , Ω , H are defined in `Core.lean` with values identical to the Python dictionaries in `algebra.py`.

The four axioms A–D are stated as `axiom` declarations in `Core.lean` and are used as hypotheses in derived theorems. Selected lattice identities that have been formally proved include:

- $\Phi_c \sqcap x = \Phi_c$ for all x (absorbing, meet direction).
- $\Phi_c \sqcup x = \Phi_c$ for all x (absorbing, join direction).
- $d(A, A) = 0$ for all A (reflexivity of canonical distance).
- Idempotency: $A \sqcap A = A$ and $A \sqcup A = A$.

The path-dependence theorem is stated as a proposition in the Lean source and the proof sketch of §6.3 is its informal counterpart; full formal proof is in progress.

0.11 Conclusion

We have presented a 12-primitive constraint algebra for substrate-independent structural comparison. The grammar assigns every system a synthon — a tuple drawn from fixed value sets — and equips the space of synthons with three operations: meet ($\sqcap$), join ($\sqcup$), and tensor product ($\otimes$). The operations are grounded in lattice theory: meet and join are the standard bounded lattice operations on the product of ordered and discrete posets; tensor is join with a stricter physical precondition.

The key properties are:

1. **Categorical primitives** admit only exact agreement or CONFLICT; ordered primitives resolve by min (meet) or max (join/tensor).
2. Φ_c **is absorbing** under all lattice operations: criticality propagates unconditionally in any composition.
3. **The path-dependence theorem** formalises catalysis, scope promotion, and templating as the single algebraic phenomenon of a context A providing a value that enables the $B \rightarrow C$ transition.
4. **Four cross-primitive axioms** constrain the valid synthon space and are machine-checked in Lean 4.
5. **The canonical Hamming distance** is kernel-verified and coincides with the Lean `primitiveMismatches` function.

The grammar is substrate-independent by construction: its primitives are defined by their structural role, not by their physical realisation. Any system that admits a description in terms of dimensionality, connectivity, recognition, polarity, fidelity, kinetics, granularity, interaction grammar, criticality, chirality, stoichiometry, and topological protection can be placed in the same metric space and compared via the operations defined here. Applications to physical systems — proteins, gauge fields, quantum materials, condensates, cosmological structures — are the subject of Paper 2.

0.12 References

- [1] P. Gärdenfors, *Conceptual Spaces: The Geometry of Thought*, MIT Press, 2000.
- [2] B. Ganter and R. Wille, *Formal Concept Analysis: Mathematical Foundations*, Springer, 1999.
- [3] S. Mac Lane, *Categories for the Working Mathematician*, 2nd ed., Springer, 1998.
- [4] G. Birkhoff, *Lattice Theory*, 3rd ed., American Mathematical Society, 1967.

-
- [5] L. de Moura and S. Ullrich, "The Lean 4 Theorem Prover and Programming Language," in *CADE-28*, Lecture Notes in Computer Science vol. 12699, Springer, 2021.
 - [6] T. Nipkow, L. C. Paulson, and M. Wenzel, *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*, Springer, 2002. (For context on interactive proof assistants.)
 - [7] D. Wiedijk (ed.), *The Seventeen Provers of the World*, Lecture Notes in Artificial Intelligence vol. 3600, Springer, 2006.
 - [8] B. A. Davey and H. A. Priestley, *Introduction to Lattices and Order*, 2nd ed., Cambridge University Press, 2002.
 - [9] E. Riehl, *Category Theory in Context*, Dover, 2016. (Open access at emilyriehl.net.)
 - [10] P. G. Ciarlet, *Linear and Nonlinear Functional Analysis with Applications*, SIAM, 2013. (For the metric space and normed space background.)

SynthOmnicon Primitive Grammar · Paper 1 of 2 · v0.4.57 · 2026-03-27 Implementation: `synthomnicon/algebra.py`, `synthomnicon/models.py` · Verification: `Synthon.lean`, `Core.lean`